Process Rubric

Project: A bilingual website and AI legal concierge for Klishin & Partners (Moscow bar association) · **Course:** MSc-HEI2058 AI in Hospitality, Entrepreneurship and Innovation · **Student:** Maksim [Surname] · **Lecturer:** Paolo Montemurro · **Hand-in:** 14.10.2026
Live outputs: https://lawfirmrussia.com (RU/EN site with assistant) · https://t.me/lawalertsbot (Telegram assistant) · https://github.com/MarsmaxO-0/glion_test_project_maksim_16.09.2026 (code, branch claude/new-session-q98ffc)
Tools: Claude design canvas (prototype), ChatGPT (image generation), Claude Code with Claude Fable 5.1 (agentic build and deployment), Claude API claude-opus-5-5 (the assistant inside the product), DigitalOcean, GoDaddy, Let's Encrypt, Cloudflare Turnstile, Telegram Bot API, Google PageSpeed Insights, ApacheBench.

# Step	Goal	Tool	Prompt (or summary)	Output	Iterations / what changed	Manual edits
1	Choose the business and define the artifact	Own research; Claude	*Reviewed the public site klishin.ru of a real Moscow bar association; decided on a bilingual redesign plus an AI assistant that routes visitors to the right practice and advocate.*	Scope: 7 sections, 15 biographies in RU/EN, practices → advocates mapping, chat assistant, contact form, deployment on a real domain.	Initial idea was a redesign only; the assistant and Telegram channel were added after reading the assignment's indicative directions ("AI concierge chatbot", "multilingual assistant").	Selection of business and features by the student.
2	Design an interactive prototype of every page	Claude (design canvas)	*Design a modern bilingual website for the law firm Klishin & Partners: home, practices, team with biographies, track record, IP centre, press centre, contacts; Russian and English; use the firm's published content.*	Single-file HTML prototype (all pages, RU and EN), bio data extracted into JSON, press feed logic, design tokens (navy, terracotta, serif headings).	Several rounds on layout and copy inside the design tool; the chosen variant was exported as a zip.	None to the generated files; selection among variants.
3	Create a hero illustration	ChatGPT (image/GIF generation)	*Generate an animated line illustration of Themis (scales of justice) in a light outline style on a dark navy background, suitable as a website hero background.*	Two animated GIFs (0.9 MB and 1.6 MB).	Second variant chosen for composition; later found too heavy (1.6 MB, 295 frames) and converted to MP4/WebM in step 17.	Choice of variant; files uploaded to the repository.
4	Put the prototype under version control	Claude Code	*[Uploaded the design zip] — the agent inspected it and imported it into the GitHub repository.*	First commit with the prototype, README describing the files.	Hidden preview cache dropped; unused upload images kept for reference.	None.

# Step	Goal	Tool	Prompt (or summary)	Output	Iterations / what changed	Manual edits
5	Rebuild the site as a real web application	Claude Code (Fable 5.1)	*"Create this website using the following technologies: django python html css js, sqlite."*	Django 6 project: 9 content models with paired RU/EN fields, views, templates, CSS/JS, language-prefixed URLs, admin, contact form, extraction script that turned the prototype into a 61-record fixture, 10 tests.	Extraction script needed four fixes (press items, IP areas, case order, settings quoting). English practice summaries did not exist in the prototype and were written by the model. Visual check in a headless browser; a scroll-reveal fallback added after sections rendered blank in screenshots.	None to code; student reviewed the English summaries.
6	See the result in a browser without a local setup	Claude Code → claude.ai artifact	*"want to check site in browser" / "want to see through localhost 8000"*	Static export of all 44 pages published as a private artifact; explanation that localhost is unreachable from the cloud.	Artifact republished later with the chat widget wired to the claude.ai viewer so the bot could be tried without a server.	None.
7	Add an AI assistant that knows the site and basic Russian law	Claude Code; Claude API (claude-opus-5-5)	*"Integrate a chat with the knowledge of the website and Russian legislation (short terms)... where is located what people can get on this site and in general the context of the website. I will use Claude key."*	Knowledge builder that renders the database into a system prompt; bilingual legal reference (codes, courts, deadlines); rules (no case-specific advice, link to real pages); chat widget; /api/chat/ endpoint with prompt caching and refusal fallback; ChatLog model; 6 tests with the API mocked.	Logging added after a "502 Bad Gateway" hid the real cause; later the Telegram bot reused the same module.	Student created the Claude API key and configured it on the machine and server; checked the legal reference.
8	Run the project on the student's Mac	Claude Code (guidance); Terminal	*"how to run these commands" / screenshots of errors*	Step-by-step instructions; diagnosis of errors from screenshots.	Three problems solved in turn: private repository blocked git clone (zip download instead); system Python too old for Django 6 (installed 3.13, recreated venv); API key pasted into the running server window instead of the shell; key in the wrong format.	All commands typed by the student; Python installed by the student.

# Step	Goal	Tool	Prompt (or summary)	Output	Iterations / what changed	Manual edits
9	Make lawyer photos manageable without klishin.ru	Claude Code	*"Can photos be uploaded in the admin instead of linked from klishin.ru?" → "do it" → "I'll upload them myself later; show default silhouettes."*	ImageField with preview in the admin, media serving, fetch_photos command, silhouettes by default.	Automatic download failed: klishin.ru returns 403 to the server even with browser headers, so the design changed to manual upload.	Photos to be uploaded by the student via the admin.
10	Deploy to a public server	Claude Code; DigitalOcean Ubuntu 24.04 droplet	*"I want deploy here" [screenshot of the droplet]*	deploy/setup.sh: packages, clone, venv, migrations, content, collectstatic, gunicorn systemd service, nginx; secrets in /etc/klishin.env; deploy/README.md.	Four re-runs to fix: gunicorn socket path not writable by www-data (502), git "dubious ownership" on re-run, health check hitting a host nginx rejects, duplicate gzip directive. Each diagnosed from a console screenshot.	Student made the repository public, ran the installer, edited the env file in nano.
11	Connect a domain and HTTPS	GoDaddy DNS; certbot; Claude Code guidance	*"hey lets put [lawfirmrussia.com]" [GoDaddy screenshot]*	A record pointed at the server; ALLOWED_HOSTS and CSRF origins set; Let's Encrypt certificate with auto-renewal; HTTP→HTTPS redirect.	DNS propagation checked with dig against GoDaddy's nameserver before issuing the certificate; installer changed to keep certbot's nginx config on later runs.	Student edited DNS in GoDaddy and ran certbot interactively.
12	Operations: spam protection, backups, monitoring, error pages	Claude Code; Telegram Bot API	*"Let's do the technical items first: captcha, automatic backups to the cloud, 404 and maintenance pages, uptime monitoring."*	Honeypot + arithmetic captcha; daily backup (SQLite online backup + media) delivered to Telegram, 14 kept; watchdog every 2 minutes; maintenance middleware; styled 404/500; /healthz/; Telegram notification for each enquiry.	Watchdog produced false alarms every two minutes (nginx 404 after HTTPS, state file wiped by systemd); fixed by checking the gunicorn socket directly and moving the state file. Backup test deadlocked on the in-memory test DB; command now dumps SQL in tests.	Student created the Telegram bot in BotFather and set token and chat id on the server.
13	Replace the captcha with something harder to fool	Claude Code; Cloudflare Turnstile	*"Can the captcha be a puzzle you drag?" → after testing: "the slider says passed even when it is wrong; use a third-party service instead."*	Slide-puzzle captcha (server-rendered image, session-stored answer) built and tested; then removed in favour of Cloudflare Turnstile verified server-side.	Two implementations discarded. A side bug found by the student: a script tag had leaked into the page title; template rewritten.	Student created the Turnstile widget and keys; tested on phone and desktop.

# Step	Goal	Tool	Prompt (or summary)	Output	Iterations / what changed	Manual edits
14	Put the assistant into Telegram and link it from the site	Claude Code; Telegram Bot API	*"Let's make it so people can go from the site to a Telegram bot."*	telegram_bot management command (long polling, per-chat memory, language detection, rate limit, /start /contacts /new, Markdown→Telegram HTML); klishin-bot systemd service; links in the chat header, contacts page and footer.	Language detection initially treated "/start" as English; fixed. Bot username and SITE_URL set on the server.	Student configured the bot's name and commands in BotFather.
15	Test with real devices and collect evidence	Phone, Mac, Telegram	*Student used the site, the form and both bots as a visitor.*	Form submissions delivered to Telegram; bot conversations logged; backup archive received.	Found: form felt slow after submit (Telegram call was synchronous; moved to a background thread), bot had no "Start" hint (commands menu added), site slow on phone (step 16–17).	Student's observations drove each fix.
16	Diagnose slow loading and size the server	DigitalOcean insights; ApacheBench; Claude Code	*"Why can the site load slowly?" / "what plan should I buy?" / "can we test the load?"*	Memory at 83% on the 512 MB plan; resized to 1 GB. Load test: 500 requests at concurrency 20, 0 failures, 49 req/s, 20 ms per page.	Installer also gained swap and lighter gunicorn settings; after resize the droplet had to be powered on manually.	Student resized the droplet and ran ab on the server.
17	Front-end performance	Claude Code; ffmpeg; PageSpeed Insights	*[PageSpeed screenshot: 92 mobile, render-blocking fonts]*	Hero GIF (1.6 MB) → MP4 128 KB / WebM 138 KB with poster; gzip and HTTP/2 in nginx; Google Fonts loaded without blocking first paint.	Score stayed at 92 (remaining cost is network distance Frankfurt→Russia); Cloudflare CDN proposed with step-by-step instructions, left for the student to decide.	Student ran PageSpeed before and after.
18	Search indexation and sharing	Claude Code	*Add what is needed for Google indexation and link previews.*	sitemap.xml (static pages and all biographies in both languages), robots.txt, hreflang and OpenGraph tags.	Submission to Google Search Console pending (student action).	—

# Step	Goal	Tool	Prompt (or summary)	Output	Iterations / what changed	Manual edits
19	Produce the assessment deliverables	Claude Code; claude.ai Slides	*[Uploaded the project outline] "Now we need to do this: pitch (max 12 slides) and process rubric."*	12-slide pitch with live links and the four reflections; this rubric generated from the session log with the real sequence of prompts, errors and fixes.	Student to fill bracketed placeholders (surname, real-world engagement outcomes) and sign the statement.	Review and edits by the student before submission.

All code, copy and documentation were AI-generated under the student's direction and reviewed by the student; the student performed every action on external services (accounts, keys, DNS, server commands, device testing) and made all accept/reject decisions. Commit history (27 commits, 01:07–05:20 on 14.10.2026) is available in the repository as a time-stamped record.

Appendix A — Evidence of real-world engagement (to be completed by the student)

- Public deployment: https://lawfirmrussia.com (HTTPS, own domain) — live since 14.10.2026.
- Form submissions received in Telegram: [insert screenshot(s)].
- Assistant conversations (site and Telegram) from the admin "Chat logs": [insert count and example].
- Response from Klishin & Partners after presenting the site: [date, channel, summary].
- Feedback from users outside the course: [N people, main comments].
- Google Search Console: sitemap submitted on [date]; indexed pages [N].

Statement of authorship

I certify that I am the author of this original, previously unpublished piece of work. I have clearly quoted, referenced and acknowledged any sources using current APA style referencing conventions. AI use is allowed for this project; I have clearly acknowledged all AI-generated or translated text using current APA style referencing conventions, and this Process Rubric serves as the appendix detailing my inputs, the AI tools' outputs, and how I used the AI-generated material in my work. I am aware that it is a breach of GIHE regulations to copy the work of another without clear acknowledgement and that attempting to do so renders me liable to disciplinary procedures. I have uploaded my work onto Turnitin and have ensured that I have made any relevant corrections to my work prior to submission.

Name: Maksim [Surname] | Signature | Date: 14.10.2026

AI tools used (APA-style acknowledgement): Anthropic. (2026). *Claude Code* [AI agent, model Claude Fable 5.1]; Anthropic. (2026). *Claude* [design canvas]; Anthropic. (2026). *Claude API* [claude-opus-5-5, used inside the product]; OpenAI. (2026). *ChatGPT* [image generation].